Redes convolucionais: deep learning para imagens

Rafael Stutz

Aula 4 · 08/10/2026

Deep Learning (DEPAI) · IASEG

Roteiro: o que faremos hoje

1. Imagens como números: pixels, canais e formatos;
2. Por que uma rede densa não aproveita bem a estrutura da imagem;
3. Convolução: filtros, mapas, preenchimento e parâmetros;
4. Max pooling: reduzir mapas sem perder tudo;
5. Montar uma CNN no Keras e interpretar o `summary`;
6. Comparar com uma rede densa e testar pixels embaralhados.

Pergunta central: como uma rede pode aprender padrões locais numa imagem?

Abertura

Onde estamos

Aula 1 · 05/10 O curso; história do deep learning; redes neurais	**Aula 2** · 06/10 Revisão; viés, variância e ruído; atalhos e vazamento; Keras	**Aula 3** · 07/10 Funções de ativação e de perda; taxa de aprendizado, lote e época	**Aula 4** · 08/10 Imagens, convoluções, pooling e CNNs no Keras
Aula 5 · 19/10 Generalização, aumentação, transferência e Grad-CAM	**Aula 6** · 20/10 Séries temporais: janelas, referências, RNN e LSTM	**Aula 7** · 21/10 Texto: embeddings, atenção, Transformers e LLMs	**Entrega** · 22/10 Apresentação do projeto e perguntas

Nas Aulas 1 a 3, a entrada da rede era uma **linha de tabela**. Hoje, a entrada é uma imagem.

O que vocês já precisam saber

Da Aula 1

- Para o computador, uma imagem é um monte de **números** (pixels);
- Cada camada monta algo mais abstrato: bordas, traços, letras;
- 2012: uma rede **convolucional** vence o ImageNet.

Da Aula 2

- Camada densa: cada neurônio recebe **todas** as entradas;
- Parâmetros: $(\text{entradas} + 1) \times \text{neurônios}$;
- Sobreajuste e atalhos (o husky e a neve).

Da Aula 3

- ReLU nas camadas ocultas;
- Saída e perda: a tabela da tarefa;
- Adam com taxa 0,001; lote 32; `EarlyStopping`.

Alguma dessas frases não está clara? É a hora de perguntar.

Para começar

Para discutir

1. Como vocês explicariam a um computador, com regras, o que é uma **moto** numa foto?
2. A regra funciona de dia e de noite? De frente e de lado? Perto e longe da câmera?

Para começar

Para discutir

1. Como vocês explicariam a um computador, com regras, o que é uma **moto** numa foto?
2. A regra funciona de dia e de noite? De frente e de lado? Perto e longe da câmera?

- “Duas rodas”: e se uma roda estiver escondida? “Um guidão”: como achar um guidão nos pixels?
- É o problema dos dígitos da Aula 1: **as regras não acabam**, e cada foto nova quebra alguma;
- Hoje: uma rede que aprende a olhar imagens, a partir de exemplos.

Imagens na segurança pública



Tarefa	Tipo de saída (Aula 3)
Ler os caracteres de uma placa	
Tipo de veículo na câmera	
Há pessoa? veículo? moto?	
A cédula é falsa?	
Dígito escrito num formulário	

Para discutir

Que tipo de saída tem cada tarefa? Em qual vocês já viram um sistema funcionando?

Imagens na segurança pública



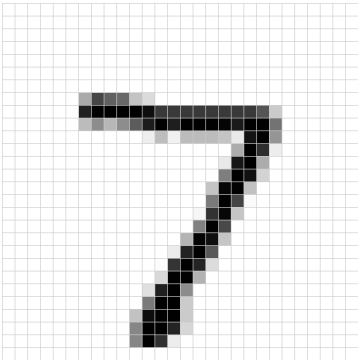
Tarefa	Tipo de saída (Aula 3)
Ler os caracteres de uma placa	multiclasse, por caractere
Tipo de veículo na câmera	multiclasse
Há pessoa? veículo? moto?	multirrótulo
A cédula é falsa?	binária
Dígito escrito num formulário	multiclasse (10 classes)

Para discutir

Que tipo de saída tem cada tarefa? Em qual vocês já viram um sistema funcionando?

O que o computador vê

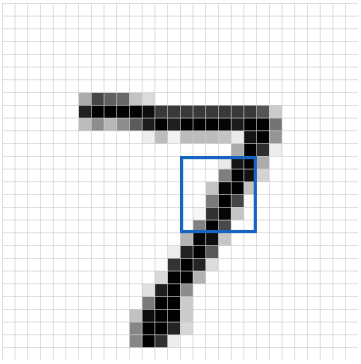
Uma imagem é uma tabela de números



Um 7 do MNIST: 28×28 pixels

O primeiro dígito do conjunto de teste do MNIST. Valores reais.

Uma imagem é uma tabela de números



Um 7 do MNIST: 28 × 28 pixels

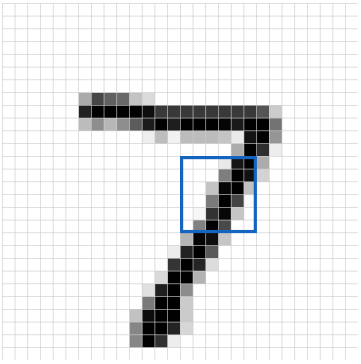
0	0	0	22	233	255
0	0	0	129	254	238
0	0	59	249	254	62
0	0	133	254	187	5
0	9	205	248	58	0
0	126	254	182	0	0

a região marcada, ampliada

- Cada **pixel** é um número: 0 = sem traço; 255 = traço intenso. A figura inverte as cores;

O primeiro dígito do conjunto de teste do MNIST. Valores reais.

Uma imagem é uma tabela de números



Um 7 do MNIST: 28 × 28 pixels

0	0	0	22	233	255
0	0	0	129	254	238
0	0	59	249	254	62
0	0	133	254	187	5
0	9	205	248	58	0
0	126	254	182	0	0

a região marcada, ampliada

- Cada **pixel** é um número: 0 = sem traço; 255 = traço intenso. A figura inverte as cores;
- $28 \times 28 = 784$ números. A rede não recebe “um 7”: recebe 784 números.

O primeiro dígito do conjunto de teste do MNIST. Valores reais.

Agora vocês: que letra é esta?

0	1	1	1	1	0
1	1	0	0	1	1
1	1	0	0	0	0
1	1	0	0	0	0
1	1	0	0	0	0
1	1	0	0	1	1
0	1	1	1	1	0

1 = tinta, 0 = papel

1. Olhem só os números: que letra é esta?
2. Quantos números há na tabela?

Agora vocês: que letra é esta?

0	1	1	1	1	0
1	1	0	0	1	1
1	1	0	0	0	0
1	1	0	0	0	0
1	1	0	0	0	0
1	1	0	0	1	1
0	1	1	1	1	0

1 = tinta, 0 = papel

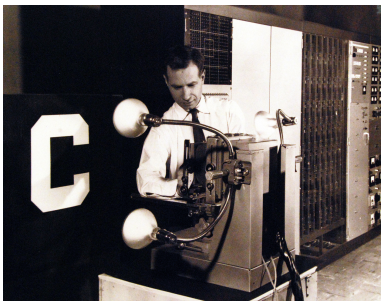
1. Olhem só os números: que letra é esta?
2. Quantos números há na tabela?

Resposta: um **C**, com $7 \times 6 = 42$ números.
Pintando os 1, a letra aparece.

Agora vocês: que letra é esta?

0	1	1	1	1	0
1	1	0	0	1	1
1	1	0	0	0	0
1	1	0	0	0	0
1	1	0	0	0	0
1	1	0	0	1	1
0	1	1	1	1	0

1 = tinta, 0 = papel



Em 1958, o perceptron (Aula 1) via letras assim: uma câmera de $20 \times 20 = 400$ fotocélulas, **400 números**.

Imagens coloridas: três tabelas



- Numa imagem colorida, cada pixel tem **três números**: quanto de **vermelho**, de **verde** e de **azul** (RGB), de 0 a 255;
- São três tabelas do mesmo tamanho, uma por cor: os **canais**.

Imagens coloridas: três tabelas



- Numa imagem colorida, cada pixel tem **três números**: quanto de **vermelho**, de **verde** e de **azul** (RGB), de 0 a 255;
- São três tabelas do mesmo tamanho, uma por cor: os **canais**.

Pixel	Vermelho (R)	Verde (G)	Azul (B)
faixa azul	0	51	153
fundo branco	250	250	250
letra	20	20	20

Valores lidos na própria figura.

Palpite: a faixa azul

Palpite, antes de ver o resultado

Separamos a placa nas três tabelas. Em qual delas a faixa azul aparece **clara**? E as letras?

Palpite: a faixa azul

Palpite, antes de ver o resultado

Separamos a placa nas três tabelas. Em qual delas a faixa azul aparece **clara**? E as letras?



vermelho (R)



verde (G)



azul (B)

Palpite: a faixa azul

Palpite, antes de ver o resultado

Separamos a placa nas três tabelas. Em qual delas a faixa azul aparece **clara**? E as letras?



vermelho (R)



verde (G)



azul (B)

- Faixa azul $(0, 51, 153)$: **preta** no vermelho, escura no verde, clara no azul;
- Letras $(20, 20, 20)$: escuras nas três. Fundo branco $(250, 250, 250)$: claro nas três.

Agora vocês: quantos números?

$$\text{números} = \text{altura} \times \text{largura} \times \text{canais}$$

Imagem	Tamanho	Canais	Números
Dígito do MNIST	28×28	1 (cinza)	
A placa desta aula	240×500	3 (cor)	
Câmera de rua (Full HD)	1080×1920	3	
Foto de celular	3000×4000	3	

Agora vocês: quantos números?

$$\text{números} = \text{altura} \times \text{largura} \times \text{canais}$$

Imagem	Tamanho	Canais	Números
Dígito do MNIST	28×28	1 (cinza)	784
A placa desta aula	240×500	3 (cor)	
Câmera de rua (Full HD)	1080×1920	3	
Foto de celular	3000×4000	3	

Agora vocês: quantos números?

$$\text{números} = \text{altura} \times \text{largura} \times \text{canais}$$

Imagem	Tamanho	Canais	Números
Dígito do MNIST	28×28	1 (cinza)	784
A placa desta aula	240×500	3 (cor)	360.000
Câmera de rua (Full HD)	1080×1920	3	
Foto de celular	3000×4000	3	

Agora vocês: quantos números?

$$\text{números} = \text{altura} \times \text{largura} \times \text{canais}$$

Imagem	Tamanho	Canais	Números
Dígito do MNIST	28×28	1 (cinza)	784
A placa desta aula	240×500	3 (cor)	360.000
Câmera de rua (Full HD)	1080×1920	3	6.220.800
Foto de celular	3000×4000	3	

Agora vocês: quantos números?

$$\text{números} = \text{altura} \times \text{largura} \times \text{canais}$$

Imagem	Tamanho	Canais	Números
Dígito do MNIST	28×28	1 (cinza)	784
A placa desta aula	240×500	3 (cor)	360.000
Câmera de rua (Full HD)	1080×1920	3	6.220.800
Foto de celular	3000×4000	3	36.000.000

Um quadro de câmera de rua tem mais de **6 milhões** de números. Guardem esse número para o próximo bloco.

Pergunta: o tamanho da entrada

Uma câmera antiga grava imagens coloridas de 480×640 pixels. Quantos números há em cada imagem?

a) 307.200;

b) 921.600;

c) 1.123;

d) 3.

Pergunta: o tamanho da entrada

Uma câmera antiga grava imagens coloridas de 480×640 pixels. Quantos números há em cada imagem?

a) 307.200;

b) 921.600;

c) 1.123;

d) 3.

Resposta: b. $480 \times 640 = 307.200$ pixels, e cada um tem 3 números (R, G e B): $307.200 \times 3 = 921.600$. A opção (a) esquece as cores; (c) soma em vez de multiplicar.

Lembram das cédulas falsas?

Aula 2: a rede recebeu 4 números

- *variance*, *skewness*, *kurtosis*, *entropy*;
- Alguém escolheu **o que medir** na imagem da cédula, e escreveu as contas.

Hoje: a rede recebe a imagem

- Todos os pixels, sem contas prévias;
- A rede **aprende o que medir**: as representações da Aula 1.

Para discutir

Que vantagem e que desvantagem vocês veem em deixar a rede escolher o que medir?

Lembram das cédulas falsas?

Aula 2: a rede recebeu 4 números

- *variance*, *skewness*, *kurtosis*, *entropy*;
- Alguém escolheu **o que medir** na imagem da cédula, e escreveu as contas.

Hoje: a rede recebe a imagem

- Todos os pixels, sem contas prévias;
- A rede **aprende o que medir**: as representações da Aula 1.

Para discutir

Que vantagem e que desvantagem vocês veem em deixar a rede escolher o que medir?

Vantagem: não depende de alguém saber o que medir, e costuma acertar mais.
Desvantagem: precisa de mais exemplos, e é mais difícil saber o que a rede olhou (Aula 5).

Por que não uma rede densa?

Uma rede densa para imagens

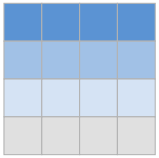
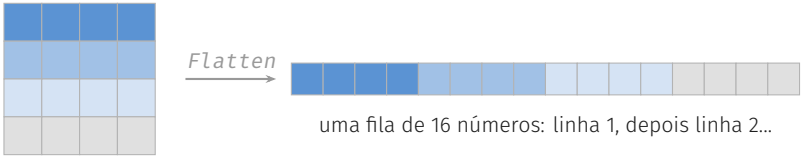


imagem 4×4

Uma rede densa para imagens



- A camada *`Flatten`* (“achatar”) põe as linhas da imagem uma depois da outra;

Uma rede densa para imagens



- A camada *Flatten* (“achatar”) põe as linhas da imagem uma depois da outra;
- Um dígito do MNIST vira uma fila de **784 números**. Daí em diante, é a rede da Aula 3;

Uma rede densa para imagens



- A camada *Flatten* (“achatar”) põe as linhas da imagem uma depois da outra;
- Um dígito do MNIST vira uma fila de **784 números**. Daí em diante, é a rede da Aula 3;
- Funciona: no MNIST, uma rede densa acerta cerca de 97% dos dígitos. Então, qual é o problema?

Problema 1: parâmetros demais

Cada neurônio de uma camada densa tem **um peso por entrada**, mais um bias (Aula 2).

Camada	Conta	Parâmetros
`Dense(128)`, com 784 entradas *`Dense(10)`*, com 128 entradas		
	total:	

Problema 1: parâmetros demais

Cada neurônio de uma camada densa tem **um peso por entrada**, mais um bias (Aula 2).

Camada	Conta	Parâmetros
`Dense(128)`, com 784 entradas *`Dense(10)`*, com 128 entradas	$(784 + 1) \times 128$	100.480
	total:	

Problema 1: parâmetros demais

Cada neurônio de uma camada densa tem **um peso por entrada**, mais um bias (Aula 2).

Camada	Conta	Parâmetros
`Dense(128)`, com 784 entradas	$(784 + 1) \times 128$	100.480
`Dense(10)`, com 128 entradas	$(128 + 1) \times 10$	1.290
	total:	**101.770**

Problema 1: parâmetros demais

Cada neurônio de uma camada densa tem **um peso por entrada**, mais um bias (Aula 2).

Camada	Conta	Parâmetros
`Dense(128)`, com 784 entradas	$(784 + 1) \times 128$	100.480
`Dense(10)`, com 128 entradas	$(128 + 1) \times 10$	1.290
	total:	**101.770**

E num quadro de câmera de rua, com **6.220.800** números?

Problema 1: parâmetros demais

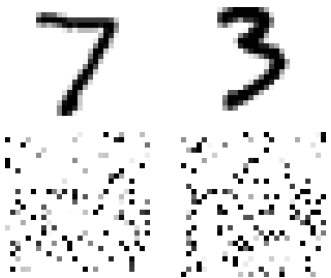
Cada neurônio de uma camada densa tem **um peso por entrada**, mais um bias (Aula 2).

Camada	Conta	Parâmetros
`Dense(128)`, com 784 entradas	$(784 + 1) \times 128$	100.480
`Dense(10)`, com 128 entradas	$(128 + 1) \times 10$	1.290
	total:	**101.770**

E num quadro de câmera de rua, com **6.220.800** números?

- Só a primeira camada: $(6.220.800 + 1) \times 128 \approx$ 796 milhões de parâmetros;
- Lento, pesado, e com tantos parâmetros a rede precisa de muitíssimos exemplos para não sobreajustar (Aula 2).

Palpite: pixels embaralhados



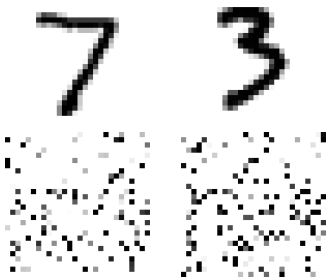
em cima: original; embaixo: embaralhado

Palpite, antes de ver o resultado

Embaralhamos os pixels de **todas** as imagens, sempre do mesmo jeito: cada pixel vai para um lugar sorteado, o mesmo em todas. Treinamos a mesma rede densa. O acerto no teste cai muito, pouco ou nada?

Nosso experimento: MNIST, 5 épocas, lote 128, Adam. Teste: 10.000 imagens.

Palpite: pixels embaralhados



em cima: original; embaixo: embaralhado

Palpite, antes de ver o resultado

Embaralhamos os pixels de **todas** as imagens, sempre do mesmo jeito: cada pixel vai para um lugar sorteado, o mesmo em todas. Treinamos a mesma rede densa. O acerto no teste cai muito, pouco ou nada?

Rede densa	Imagens normais	Imagens embaralhadas
Acerto no teste	97,3%	97,2%

Nosso experimento: MNIST, 5 épocas, lote 128, Adam. Teste: 10.000 imagens.

Para discutir: por que nada mudou?

Para discutir

Nós não reconhecemos o 7 embaralhado. Por que a rede densa acerta igual?

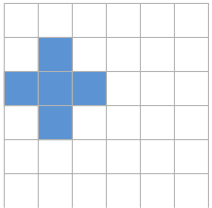
Para discutir: por que nada mudou?

Para discutir

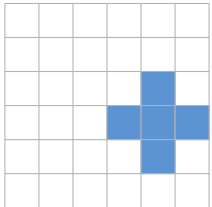
Nós não reconhecemos o 7 embaralhado. Por que a rede densa acerta igual?

- Para a camada densa, cada pixel é só **uma entrada numerada**: entrada 1, entrada 2... Ela não sabe que o pixel 30 fica **ao lado** do pixel 31;
- Trocar todos os pixels de lugar é só trocar a ordem das entradas: a rede aprende os mesmos pesos, em outra ordem;
- Nós usamos a vizinhança: traços, curvas e cantos são pixels vizinhos juntos. A rede densa **não explora diretamente a relação entre vizinhos**.

Problema 2: o mesmo objeto, em outro lugar



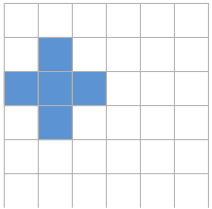
um padrão no canto



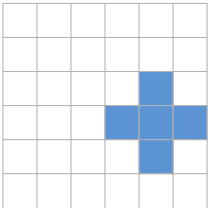
o mesmo padrão, em outro lugar

- Para a rede densa, as duas imagens acendem **entradas diferentes**, ligadas a **pesos diferentes**;
- Ela precisa aprender o padrão em cada posição, separadamente;

Problema 2: o mesmo objeto, em outro lugar



um padrão no canto



o mesmo padrão, em outro lugar

- Para a rede densa, as duas imagens acendem **entradas diferentes**, ligadas a **pesos diferentes**;
- Ela precisa aprender o padrão em cada posição, separadamente;
- Uma moto no canto e uma moto no centro ativam entradas diferentes. A rede densa pode precisar de mais exemplos em posições variadas.

O que queremos de uma rede para imagens

Para discutir

Numa foto de multidão, como vocês procuram uma pessoa de boné vermelho?

O que queremos de uma rede para imagens

Para discutir

Numa foto de multidão, como vocês procuram uma pessoa de boné vermelho?

1. Olham **um pedaço de cada vez**: pixels vizinhos, juntos;

O que queremos de uma rede para imagens

Para discutir

Numa foto de multidão, como vocês procuram uma pessoa de boné vermelho?

1. Olham **um pedaço de cada vez**: pixels vizinhos, juntos;
2. Procuram **a mesma coisa** em todos os pedaços: o mesmo detector, em todo lugar;

O que queremos de uma rede para imagens

Para discutir

Numa foto de multidão, como vocês procuram uma pessoa de boné vermelho?

1. Olham **um pedaço de cada vez**: pixels vizinhos, juntos;
2. Procuram **a mesma coisa** em todos os pedaços: o mesmo detector, em todo lugar;
3. **Juntam as partes**: boné, rosto e corpo formam uma pessoa.

O que queremos de uma rede para imagens

Para discutir

Numa foto de multidão, como vocês procuram uma pessoa de boné vermelho?

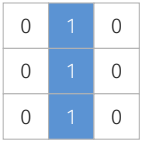
1. Olham **um pedaço de cada vez**: pixels vizinhos, juntos;
2. Procuram **a mesma coisa** em todos os pedaços: o mesmo detector, em todo lugar;
3. **Juntam as partes**: boné, rosto e corpo formam uma pessoa.

A rede convolucional (CNN)

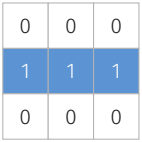
Faz exatamente essas três coisas: **filtros** pequenos (1), que percorrem a imagem inteira (2), em camadas que montam partes cada vez maiores (3).

A convolução

O filtro: um pequeno detector



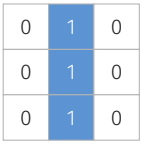
filtro **V**
(traço vertical)



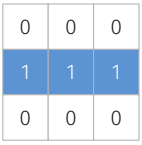
filtro **H**
(traço horizontal)

- Um **filtro** é uma tabela pequena de pesos. Aqui, 3×3: 9 pesos;

O filtro: um pequeno detector



filtro **V**
(traço vertical)



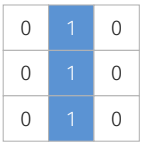
filtro **H**
(traço horizontal)

- Um **filtro** é uma tabela pequena de pesos. Aqui, 3×3: 9 pesos;
- É um neurônio da Aula 3 que olha **só 9 pixels vizinhos**: multiplica cada pixel pelo seu peso, soma e aplica a ReLU;

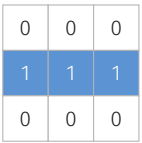
$$z = w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9 + b$$

(a mesma conta da Aula 3, com 9 entradas)

O filtro: um pequeno detector



filtro **V**
(traço vertical)



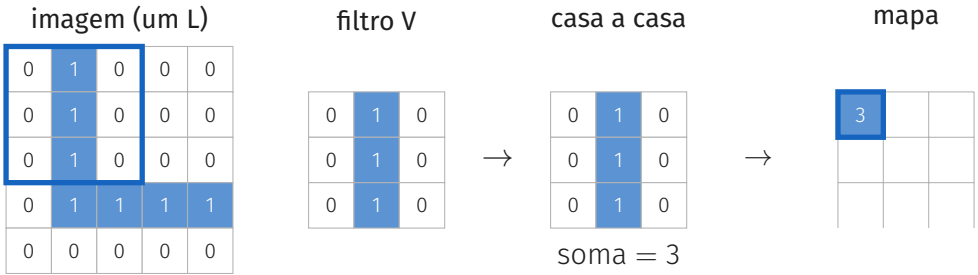
filtro **H**
(traço horizontal)

- Um **filtro** é uma tabela pequena de pesos. Aqui, 3×3: 9 pesos;
- É um neurônio da Aula 3 que olha **só 9 pixels vizinhos**: multiplica cada pixel pelo seu peso, soma e aplica a ReLU;
- O filtro **percorre** a imagem: pedaço por pedaço, com os mesmos 9 pesos.

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9 + b$$

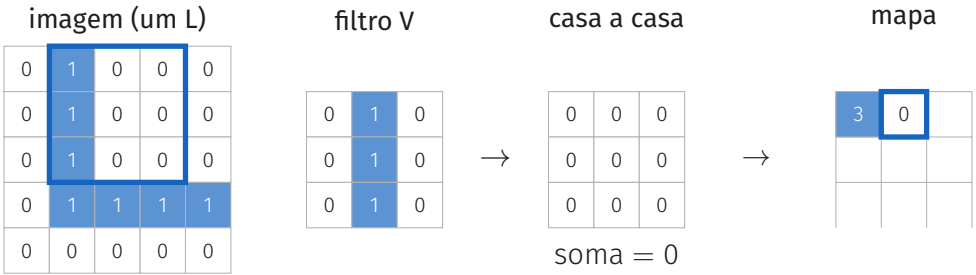
(a mesma conta da Aula 3, com 9 entradas)

A convolução, passo a passo



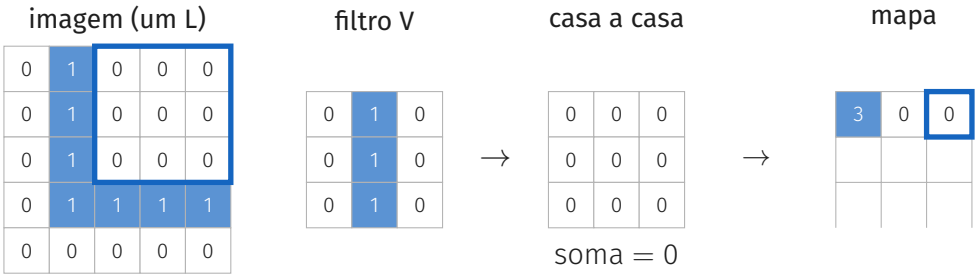
Posição 1. Coloquem o filtro sobre o canto da imagem e multipliquem casa a casa: $0{\cdot}0 + 1{\cdot}1 + 0{\cdot}0 \;+\; 0{\cdot}0 + 1{\cdot}1 + 0{\cdot}0 \;+\; 0{\cdot}0 + 1{\cdot}1 + 0{\cdot}0 = 3$. O 3 vai para o mapa.

A convolução, passo a passo



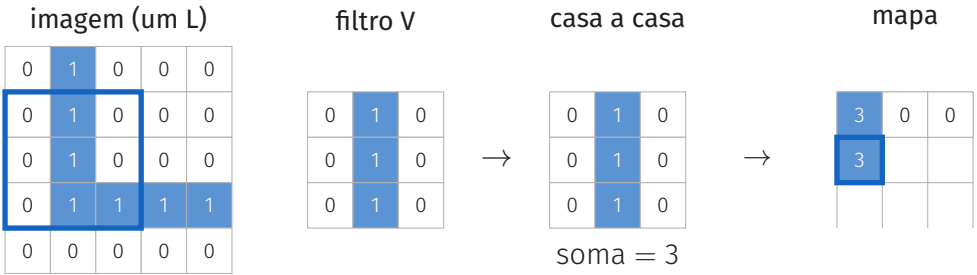
Andem uma casa para a direita e repitam. Aqui, o meio do filtro cai em casas de papel: soma 0.

A convolução, passo a passo



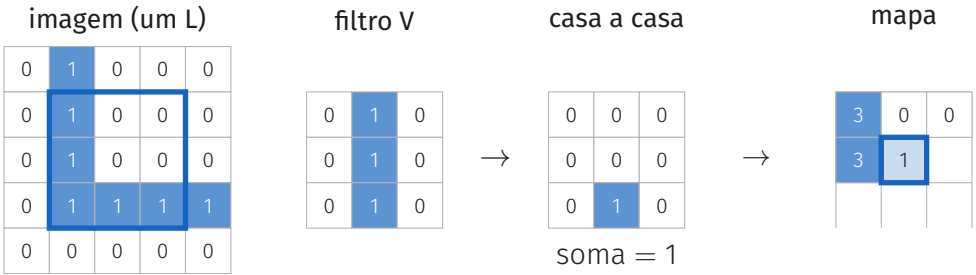
Andem uma casa para a direita e repitam. Aqui, o meio do filtro cai em casas de papel: soma 0.

A convolução, passo a passo



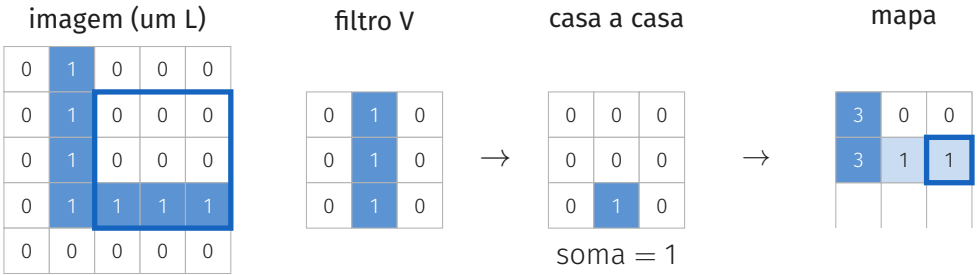
No fim da linha, **desçam uma casa** e voltem para a esquerda. Mesma conta, sempre com os mesmos 9 pesos.

A convolução, passo a passo



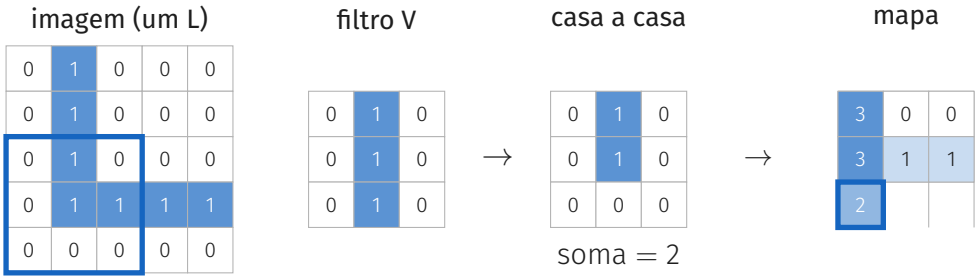
No fim da linha, **desçam uma casa** e voltem para a esquerda. Mesma conta, sempre com os mesmos 9 pesos.

A convolução, passo a passo



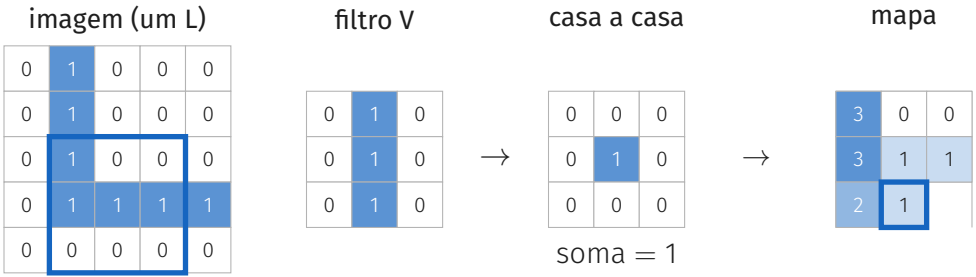
No fim da linha, **desçam uma casa** e voltem para a esquerda. Mesma conta, sempre com os mesmos 9 pesos.

A convolução, passo a passo



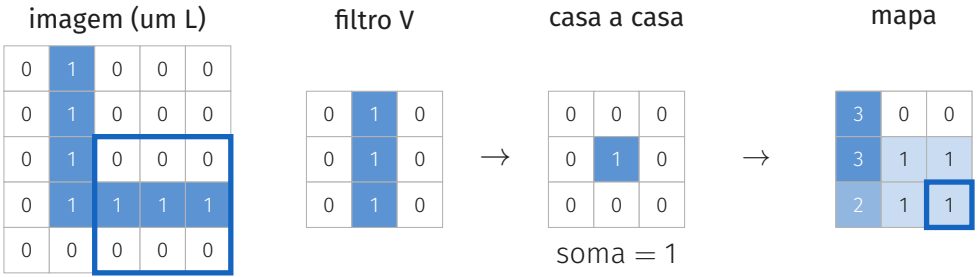
No fim da linha, **desçam uma casa** e voltem para a esquerda. Mesma conta, sempre com os mesmos 9 pesos.

A convolução, passo a passo



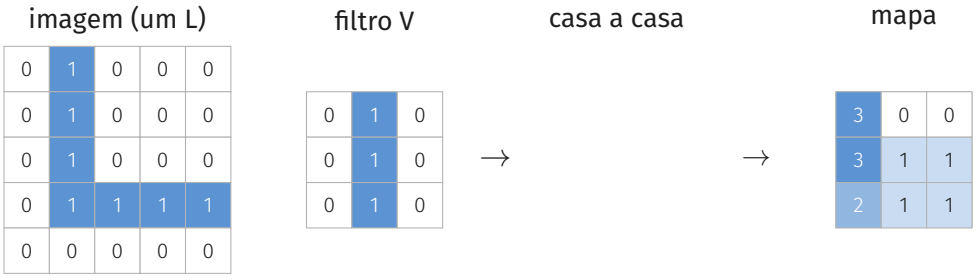
No fim da linha, **desçam uma casa** e voltem para a esquerda. Mesma conta, sempre com os mesmos 9 pesos.

A convolução, passo a passo



No fim da linha, **desçam uma casa** e voltem para a esquerda. Mesma conta, sempre com os mesmos 9 pesos.

A convolução, passo a passo



O mapa ficou alto na coluna da esquerda: é onde a imagem tem um traço vertical, a haste do L. O filtro V é um detector de traços verticais.

Agora vocês: o filtro H

imagem (o mesmo L)

0	1	0	0	0
0	1	0	0	0
0	1	0	0	0
0	1	1	1	1
0	0	0	0	0

filtro H

0	0	0
1	1	1
0	0	0

$\rightarrow$

mapa

- Calculem as 9 casas do mapa: janela 3×3, multiplicar casa a casa, somar;

Agora vocês: o filtro H



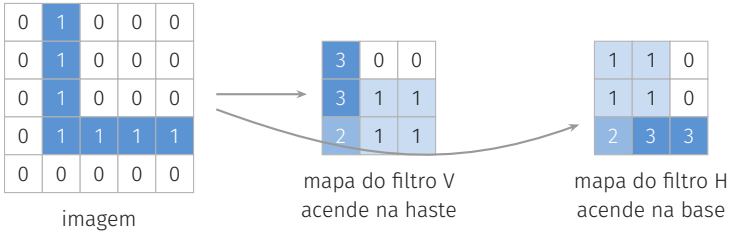
- Calculem as 9 casas do mapa: janela 3×3, multiplicar casa a casa, somar;
- Dica: com o filtro H, só a **linha do meio** da janela conta. Na posição 1, a linha do meio é $0, 1, 0$: soma 1;

Agora vocês: o filtro H



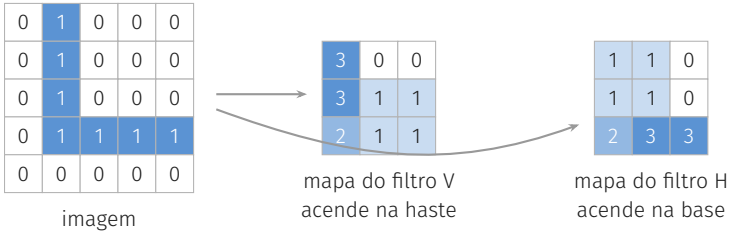
- Calculem as 9 casas do mapa: janela 3×3, multiplicar casa a casa, somar;
- Dica: com o filtro H, só a **linha do meio** da janela conta. Na posição 1, a linha do meio é $0, 1, 0$: soma 1;
- O mapa do filtro H é alto na **linha de baixo**: a base do L, o traço horizontal.

Mapas de características



- Cada filtro procura **um padrão**. O resultado é um **mapa de características**: diz onde o padrão aparece;
- Número alto: o pedaço da imagem se parece com o filtro. Número baixo: não se parece;

Mapas de características



- Cada filtro procura **um padrão**. O resultado é um **mapa de características**: diz onde o padrão aparece;
- Número alto: o pedaço da imagem se parece com o filtro. Número baixo: não se parece;
- Uma camada convolucional tem **muitos filtros**, e produz um mapa para cada um.

Agora vocês: um filtro com números negativos

imagem: papel e tinta

0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

filtro de borda

−1	0	1
−1	0	1
−1	0	1

→

mapa

- O filtro tem −1 na coluna da esquerda, 0 no meio e 1 na direita. Calculem as 4 casas do mapa;

Agora vocês: um filtro com números negativos

imagem: papel e tinta

0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

filtro de borda

−1	0	1
−1	0	1
−1	0	1

$\rightarrow$

mapa



- O filtro tem -1 na coluna da esquerda, 0 no meio e 1 na direita. Calculem as 4 casas do mapa;
- **Posição 2** (colunas 2, 3 e 4). Em cada linha: $(-1) \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 1$. Três linhas iguais: $1 + 1 + 1 = 3$;

Agora vocês: um filtro com números negativos

imagem: papel e tinta

0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

filtro de borda

−1	0	1
−1	0	1
−1	0	1

$\rightarrow$

mapa

0	3	3	0

- O filtro tem -1 na coluna da esquerda, 0 no meio e 1 na direita. Calculem as 4 casas do mapa;
- **Posição 2** (colunas 2, 3 e 4). Em cada linha: $(-1) \cdot 0 + 0 \cdot 0 + 1 \cdot 1 = 1$. Três linhas iguais: $1 + 1 + 1 = 3$;
- Onde a imagem é toda igual (só papel, ou só tinta), o -1 e o $+1$ se cancelam: **0**. Onde muda: 3. É um **detector de bordas**.

Para discutir: e ao contrário?

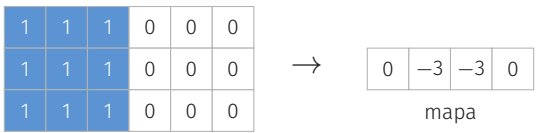
Para discutir

E se a imagem fosse tinta à esquerda e papel à direita $(1, 1, 1, 0, 0, 0)$? Que mapa sai? E depois da ReLU?

Para discutir: e ao contrário?

Para discutir

E se a imagem fosse tinta à esquerda e papel à direita $(1, 1, 1, 0, 0, 0)$? Que mapa sai? E depois da ReLU?

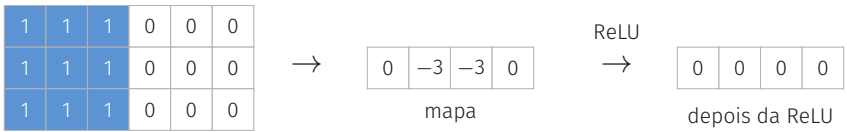


- Os sinais se invertem: -3. Depois da ReLU (negativo vira 0, Aula 3), nada acende;

Para discutir: e ao contrário?

Para discutir

E se a imagem fosse tinta à esquerda e papel à direita $(1, 1, 1, 0, 0, 0)$? Que mapa sai? E depois da ReLU?



- Os sinais se invertem: -3. Depois da ReLU (negativo vira 0, Aula 3), nada acende;
- Cada filtro detecta **uma direção** de borda. Para a outra direção, a rede aprende outro filtro, com os sinais trocados.

A mesma conta, numa foto de verdade



a placa, em cinza (1 canal)

Nas fotos, o mapa aparece sem sinal: bordas nas duas direções.

A mesma conta, numa foto de verdade



a placa, em cinza (1 canal)



filtro de borda vertical

−1	0	1
−1	0	1
−1	0	1

- O filtro do exercício, passado pela foto inteira. **Escuro** = o filtro respondeu forte;

Nas fotos, o mapa aparece sem sinal: bordas nas duas direções.

A mesma conta, numa foto de verdade



a placa, em cinza (1 canal)



filtro de borda vertical

−1	0	1
−1	0	1
−1	0	1



o mesmo filtro, deitado

−1	−1	−1
0	0	0
1	1	1

- O filtro do exercício, passado pela foto inteira. **Escuro** = o filtro respondeu forte;
- Bordas verticais: as hastes do 1, do D e do B. Bordas horizontais: a faixa azul e os traços deitados do 2 e do 3.

Nas fotos, o mapa aparece sem sinal: bordas nas duas direções.

Pergunta: qual filtro fez cada imagem?



A



B



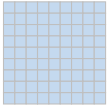
C

−1	0	1
−1	0	1
−1	0	1

1

−1	−1	−1
0	0	0
1	1	1

2



3

9×9, cada peso $= 1/81$
média dos 81 pixels

Pergunta: qual filtro fez cada imagem?



A



B



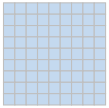
C

−1	0	1
−1	0	1
−1	0	1

1

−1	−1	−1
0	0	0
1	1	1

2



3

9×9, cada peso $= 1/81$
média dos 81 pixels

Resposta: A–2, B–3, C–1. O filtro 3 calcula a média dos 81 pixels de cada região (bias zero), borrando a imagem. A mesma operação, com pesos diferentes, produz resultados diferentes.

Quem escolhe os pesos do filtro?

- **Antes**: pessoas desenhavam os filtros à mão, como o de bordas que acabamos de usar;

Pesos reais da ResNet50 do Keras (*`weights="imagenet"`*), camada *`conv1`*.

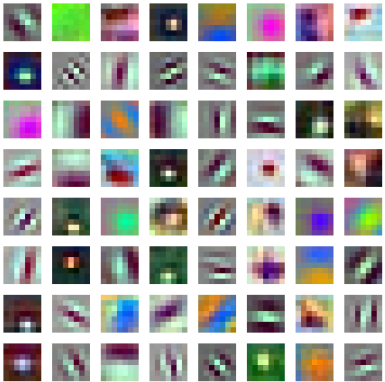
Quem escolhe os pesos do filtro?

- **Antes**: pessoas desenhavam os filtros à mão, como o de bordas que acabamos de usar;
- **Na CNN**: os pesos começam **sorteados**, e o treino os ajusta para reduzir a perda, como os pesos da Aula 3;

Pesos reais da ResNet50 do Keras (`weights="imagenet"`), camada `conv1`.

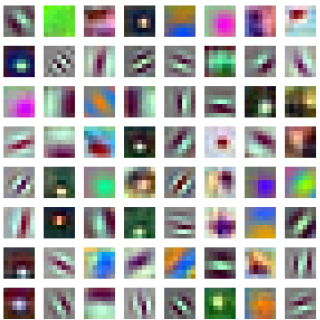
Quem escolhe os pesos do filtro?

- **Antes**: pessoas desenhavam os filtros à mão, como o de bordas que acabamos de usar;
- **Na CNN**: os pesos começam **sorteados**, e o treino os ajusta para reduzir a perda, como os pesos da Aula 3;
- Ao lado: os 64 filtros da primeira camada de uma **ResNet50**, uma CNN treinada com 1,4 milhão de fotos (ImageNet). Cada quadradinho é um filtro 7×7, colorido;
- Ninguém desenhou esses filtros: a rede os aprendeu.



Pesos reais da ResNet50 do Keras (`weights="imagenet"`), camada `conv1`.

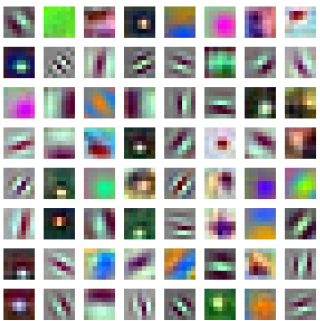
Para discutir: filtros que ninguém desenhou



Para discutir

1. O que vocês reconhecem nesses filtros?
2. Por que a rede, sozinha, chegou a detectores de borda?

Para discutir: filtros que ninguém desenhou

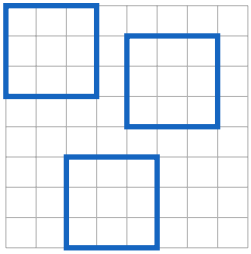


Para discutir

1. O que vocês reconhecem nesses filtros?
2. Por que a rede, sozinha, chegou a detectores de borda?

- Bordas em várias direções, listras finas e grossas, manchas de cor;
- Bordas separam um objeto do fundo: servem para **quase qualquer** tarefa com imagens. Por isso aparecem em quase toda CNN treinada. Voltamos a isso na transferência de aprendizado.

Um filtro, a imagem inteira



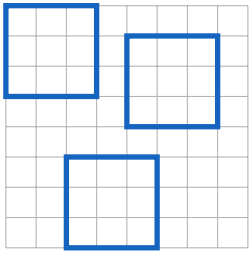
as três janelas usam
os mesmos 9 pesos

- O filtro é **o mesmo** em todas as posições: os pesos são **compartilhados**;
- Um filtro 3×3 tem 9 pesos e 1 bias: **10 parâmetros**;
- A camada densa do MNIST tinha 100.480.

Palpite, antes de ver o resultado

A imagem passa de 28×28 para 1080×1920. Quantos parâmetros tem o filtro?

Um filtro, a imagem inteira



as três janelas usam
os mesmos 9 pesos

- O filtro é **o mesmo** em todas as posições: os pesos são **compartilhados**;
- Um filtro 3×3 tem 9 pesos e 1 bias: **10 parâmetros**;
- A camada densa do MNIST tinha 100.480.

Palpite, antes de ver o resultado

A imagem passa de 28×28 para 1080×1920. Quantos parâmetros tem o filtro?

Os mesmos 10. O tamanho da imagem muda quantas vezes o filtro é aplicado, e não quantos pesos ele tem.

Palpite: o farol no canto

Palpite, antes de ver o resultado

Um filtro aprendeu a detectar um pequeno traço vertical. Se esse traço aparecer em outro lugar da imagem, o filtro também poderá detectá-lo?

Palpite: o farol no canto

Palpite, antes de ver o resultado

Um filtro aprendeu a detectar um pequeno traço vertical. Se esse traço aparecer em outro lugar da imagem, o filtro também poderá detectá-lo?

- **Sim**. O filtro percorre a imagem inteira e pode detectar o mesmo traço em diferentes posições;
- A convolução detecta o mesmo padrão **em qualquer posição**. Era o problema 2 da rede densa;
- O resto da rede ainda precisa saber usar um farol no canto. Voltamos a isso na aumentação de dados.

O tamanho do mapa



uma linha com 5 casas

o filtro de largura 3 cabe em
1 de 3 posições

Imagem	Filtro	Mapa
5×5 (o L)	3×3	
28×28 (MNIST)	3×3	
28×28	5×5	
3×6 (a borda)	3×3	

O tamanho do mapa



uma linha com 5 casas

o filtro de largura 3 cabe em
2 de 3 posições

Imagem	Filtro	Mapa
5×5 (o L)	3×3	
28×28 (MNIST)	3×3	
28×28	5×5	
3×6 (a borda)	3×3	

O tamanho do mapa



uma linha com 5 casas

o filtro de largura 3 cabe em **3 de 3** posições

Sem preenchimento

tamanho do mapa = tamanho da imagem − tamanho do filtro +1

Imagem	Filtro	Mapa
5×5 (o L)	3×3	
28×28 (MNIST)	3×3	
28×28	5×5	
3×6 (a borda)	3×3	

O tamanho do mapa



uma linha com 5 casas

o filtro de largura 3 cabe em **3 de 3** posições

Sem preenchimento

tamanho do mapa = tamanho da imagem − tamanho do filtro +1

Imagem	Filtro	Mapa
5×5 (o L)	3×3	3×3
28×28 (MNIST)	3×3	
28×28	5×5	
3×6 (a borda)	3×3	

O tamanho do mapa



uma linha com 5 casas

o filtro de largura 3 cabe em **3 de 3** posições

Sem preenchimento

tamanho do mapa = tamanho da imagem − tamanho do filtro +1

Imagem	Filtro	Mapa
5×5 (o L)	3×3	3×3
28×28 (MNIST)	3×3	26×26
28×28	5×5	
3×6 (a borda)	3×3	

O tamanho do mapa



uma linha com 5 casas

o filtro de largura 3 cabe em **3 de 3** posições

Sem preenchimento

tamanho do mapa = tamanho da imagem − tamanho do filtro +1

Imagem	Filtro	Mapa
5×5 (o L)	3×3	3×3
28×28 (MNIST)	3×3	26×26
28×28	5×5	24×24
3×6 (a borda)	3×3	

O tamanho do mapa



uma linha com 5 casas

o filtro de largura 3 cabe em **3 de 3** posições

Sem preenchimento

tamanho do mapa = tamanho da imagem − tamanho do filtro +1

Imagem	Filtro	Mapa
5×5 (o L)	3×3	3×3
28×28 (MNIST)	3×3	26×26
28×28	5×5	24×24
3×6 (a borda)	3×3	1×4

O filtro não “sai” da imagem: por isso o mapa encolhe um pouco a cada camada.

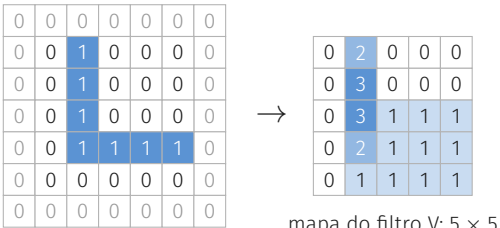
Preenchimento: uma borda de zeros

0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	0	1	0	0	0	0
0	0	1	0	0	0	0
0	0	1	1	1	1	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

o L, com uma borda de zeros

- Para o mapa não encolher, põe-se uma **borda de zeros** em volta da imagem;

Preenchimento: uma borda de zeros

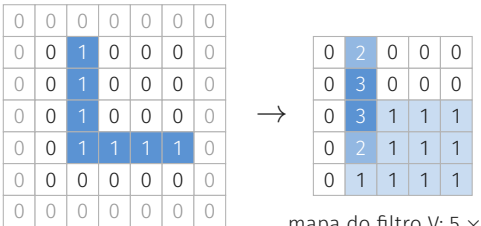


o L, com uma borda de zeros

mapa do filtro V: 5×5

- Para o mapa não encolher, põe-se uma **borda de zeros** em volta da imagem;
- 5×5 com a borda vira 7×7, e o mapa volta a ter 5×5;

Preenchimento: uma borda de zeros



o L, com uma borda de zeros

mapa do filtro V: 5×5

- Para o mapa não encolher, põe-se uma **borda de zeros** em volta da imagem;
- 5×5 com a borda vira 7×7, e o mapa volta a ter 5×5;
- No Keras:

```
layers.Conv2D(32, 3, padding="valid")
# padrao: sem borda, o mapa encolhe
layers.Conv2D(32, 3, padding="same")
# com borda: o mapa tem o mesmo
# tamanho da imagem
```

Vários filtros, vários mapas

`layers.Conv2D(32, 3, activation="relu")`: **32 filtros** de 3×3, cada um seguido da ReLU.



28 × 28 × 1

32 filtros

FIGURA mapa

FIGURA mapa

FIGURA mapa

FIGURA mapa

FIGURA mapa

FIGURA mapa

6 dos 32 mapas: a saída tem $26 \times 26 \times 32$

- A saída é uma **pilha de 32 mapas**, um por filtro. **Escuro** = o filtro acendeu ali;

Mapas reais da CNN que treinamos no MNIST (98,9% no teste).

Vários filtros, vários mapas

layers.Conv2D(32, 3, activation="relu"): **32 filtros** de 3×3, cada um seguido da ReLU.



28 × 28 × 1

32 filtros

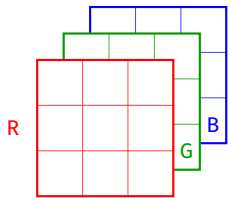
FIGURA mapa | FIGURA mapa | FIGURA mapa | FIGURA mapa | FIGURA mapa | FIGURA mapa

6 dos 32 mapas: a saída tem $26 \times 26 \times 32$

- A saída é uma **pilha de 32 mapas**, um por filtro. **Escuro** = o filtro acendeu ali;
- Cada filtro destaca uma parte diferente do 7: só o traço de cima, só a diagonal, o contorno.

Mapas reais da CNN que treinamos no MNIST (98,9% no teste).

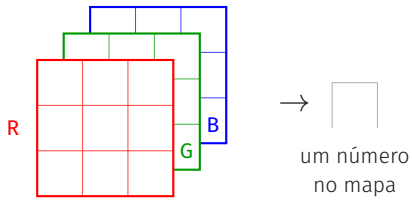
Imagens coloridas: o filtro também tem três camadas



filtro $3 \times 3 \times 3$:
uma tabela por canal

- Com 3 canais, cada filtro tem **uma tabela** 3×3 **por canal**: $3 \times 3 \times 3 = 27$ pesos, mais 1 bias;

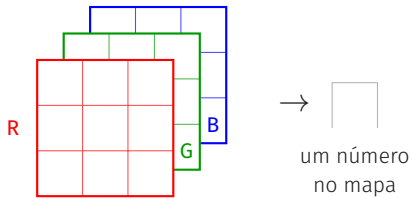
Imagens coloridas: o filtro também tem três camadas



filtro $3 \times 3 \times 3$:
uma tabela por canal

- Com 3 canais, cada filtro tem **uma tabela** 3×3 **por canal**: $3 \times 3 \times 3 = 27$ pesos, mais 1 bias;
- Multiplica casa a casa **nos três canais** e soma tudo: sai um número;

Imagens coloridas: o filtro também tem três camadas



filtro $3 \times 3 \times 3$:
uma tabela por canal

- Com 3 canais, cada filtro tem **uma tabela** 3×3 **por canal**: $3 \times 3 \times 3 = 27$ pesos, mais 1 bias;
- Multiplica casa a casa **nos três canais** e soma tudo: sai um número;
- Na segunda camada, a "imagem" é a pilha de 32 mapas: cada filtro tem $3 \times 3 \times 32$ pesos.

Contando os parâmetros de uma convolução

Parâmetros de uma camada convolucional

$$(\text{altura} \times \text{largura} \times \text{canais que entram} + 1) \times \text{filtros}$$

Camada	Entra	Conta	Parâmetros
`Conv2D(32, 3)`	um dígito (1 canal)		
`Conv2D(32, 3)`	uma foto colorida (3)		
`Conv2D(64, 3)`	32 mapas		

Contando os parâmetros de uma convolução

Parâmetros de uma camada convolucional

$$(\text{altura} \times \text{largura} \times \text{canais que entram} + 1) \times \text{filtros}$$

Camada	Entra	Conta	Parâmetros
`Conv2D(32, 3)`	um dígito (1 canal)	$(3 \times 3 \times 1 + 1) \times 32$	320
`Conv2D(32, 3)`	uma foto colorida (3)		
`Conv2D(64, 3)`	32 mapas		

Contando os parâmetros de uma convolução

Parâmetros de uma camada convolucional

$$(\text{altura} \times \text{largura} \times \text{canais que entram} + 1) \times \text{filtros}$$

Camada	Entra	Conta	Parâmetros
`Conv2D(32, 3)`	um dígito (1 canal)	$(3 \times 3 \times 1 + 1) \times 32$	320
`Conv2D(32, 3)`	uma foto colorida (3)	$(3 \times 3 \times 3 + 1) \times 32$	896
`Conv2D(64, 3)`	32 mapas		

Contando os parâmetros de uma convolução

Parâmetros de uma camada convolucional

$$(\text{altura} \times \text{largura} \times \text{canais que entram} + 1) \times \text{filtros}$$

Camada	Entra	Conta	Parâmetros
`Conv2D(32, 3)`	um dígito (1 canal)	$(3 \times 3 \times 1 + 1) \times 32$	320
`Conv2D(32, 3)`	uma foto colorida (3)	$(3 \times 3 \times 3 + 1) \times 32$	896
`Conv2D(64, 3)`	32 mapas	$(3 \times 3 \times 32 + 1) \times 64$	18.496

O tamanho da imagem (28, 1080, 1920...) **não entra na conta**. Comparem com os 100.480 da primeira camada densa.

Pergunta: parâmetros da convolução

Uma camada `Conv2D(16, 3)` recebe fotos coloridas de 100×100. Quantos parâmetros ela tem?

a) 448;

b) 160;

c) 480.016;

d) 144.

Pergunta: parâmetros da convolução

Uma camada `Conv2D(16, 3)` recebe fotos coloridas de 100×100. Quantos parâmetros ela tem?

a) 448;

b) 160;

c) 480.016;

d) 144.

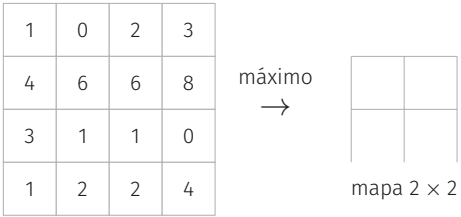
Resposta: a. $(3 \times 3 \times 3 + 1) \times 16 = 28 \times 16 = 448$. A opção (b) esquece os 3 canais: $(9 + 1) \times 16$. A opção (d) esquece os canais e o bias. A opção (c) seria de uma camada densa: o tamanho da imagem não entra na conta.

Resumo: a convolução

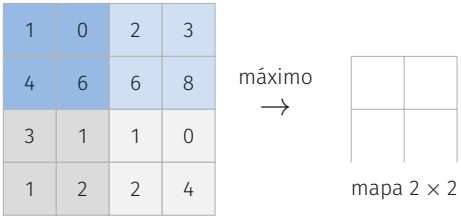
- Um **filtro** é uma tabela pequena de pesos que **percorre** a imagem: em cada posição, multiplica casa a casa e soma;
- O resultado é um **mapa de características**: onde o padrão do filtro aparece;
- Os pesos são **aprendidos** no treino, e são os mesmos em todas as posições: poucos parâmetros, e o padrão é achado em qualquer lugar;
- Sem preenchimento, o mapa encolhe: imagem $-$ filtro $+1$. Com `padding="same"`, não;
- Parâmetros: (altura $\times$ largura $\times$ canais $+ 1$) $\times$ filtros.

Pooling

Max pooling: o maior de cada bloco



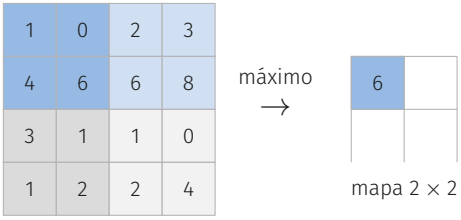
Max pooling: o maior de cada bloco



- Dividam o mapa em blocos 2×2;

Exemplo do CS231n (Stanford).

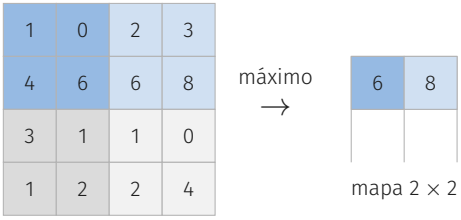
Max pooling: o maior de cada bloco



- Dividam o mapa em blocos 2×2;
- De cada bloco, fica só **o maior número**: do bloco $1, 0, 4, 6$ fica o 6;

Exemplo do CS231n (Stanford).

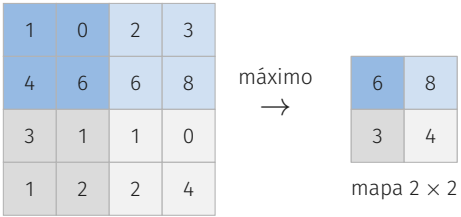
Max pooling: o maior de cada bloco



- Dividam o mapa em blocos 2×2;
- De cada bloco, fica só **o maior número**: do bloco $1, 0, 4, 6$ fica o 6;

Exemplo do CS231n (Stanford).

Max pooling: o maior de cada bloco



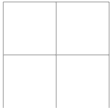
- Dividam o mapa em blocos 2×2;
- De cada bloco, fica só **o maior número**: do bloco $1, 0, 4, 6$ fica o 6;
- O mapa fica com metade da altura e metade da largura: 4 vezes menos números. O pooling **não tem pesos**: não aprende nada, só resume.

Exemplo do CS231n (Stanford).

Agora vocês: max pooling

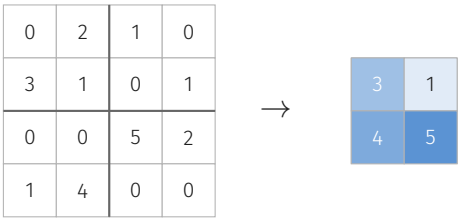
0	2	1	0
3	1	0	1
0	0	5	2
1	4	0	0

$\rightarrow$



- Blocos 2×2; de cada um, o maior;

Agora vocês: max pooling



- Blocos 2×2; de cada um, o maior;
- Resposta: $3, 1, 4, 5$. Reparem: o mapa ficou menor, mas os números altos (onde o filtro acendeu) **continuam lá**.

Palpite: o 9 andou uma casa

0	0	0	0
9	0	0	0
0	0	0	0
0	0	0	0

mapa A

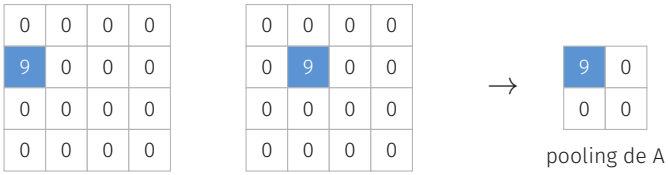
0	0	0	0
0	9	0	0
0	0	0	0
0	0	0	0

mapa B: o 9 andou uma casa

Palpite, antes de ver o resultado

O 9 andou uma casa para a direita. Depois do max pooling, os resultados de A e de B são diferentes?

Palpite: o 9 andou uma casa

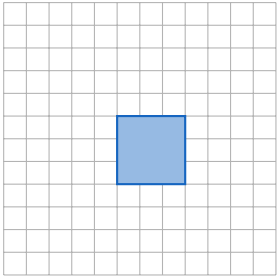


Palpite, antes de ver o resultado

O 9 andou uma casa para a direita. Depois do max pooling, os resultados de A e de B são diferentes?

Iguais: o 9 continua no mesmo bloco 2×2. O pooling tolera pequenos deslocamentos: um traço um pouco mais para o lado dá o mesmo resultado. Se o 9 andasse mais uma casa, mudaria de bloco.

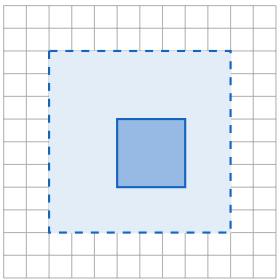
Por que encolher?



a imagem, de perto

- **Menos números**: 26×26 vira 13×13. Menos contas nas camadas seguintes;

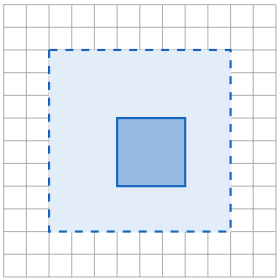
Por que encolher?



a imagem, de perto

- **Menos números**: 26×26 vira 13×13. Menos contas nas camadas seguintes;
- **A próxima camada vê mais longe**. Um número do primeiro mapa resume 3×3 pixels (quadrado cheio). Depois do pooling e de mais uma convolução, cada número resume 8×8 pixels (tracejado);

Por que encolher?



a imagem, de perto

- **Menos números**: 26×26 vira 13×13. Menos contas nas camadas seguintes;
- **A próxima camada vê mais longe**. Um número do primeiro mapa resume 3×3 pixels (quadrado cheio). Depois do pooling e de mais uma convolução, cada número resume 8×8 pixels (tracejado);
- Camada a camada, a rede passa de **traços** para **partes** e de partes para **objetos**: as três ideias do início.

Pergunta: a camada de pooling

Um mapa de $26 \times 26 \times 32$ passa por *`MaxPooling2D(2)`*. Qual é o tamanho da saída, e quantos parâmetros essa camada aprende?

a) $13 \times 13 \times 32$; nenhum;

b) $13 \times 13 \times 32$; 4 por bloco;

c) $13 \times 13 \times 16$; nenhum;

d) $52 \times 52 \times 32$; nenhum.

Pergunta: a camada de pooling

Um mapa de $26 \times 26 \times 32$ passa por *`MaxPooling2D(2)`*. Qual é o tamanho da saída, e quantos parâmetros essa camada aprende?

a) $13 \times 13 \times 32$; nenhum;

b) $13 \times 13 \times 32$; 4 por bloco;

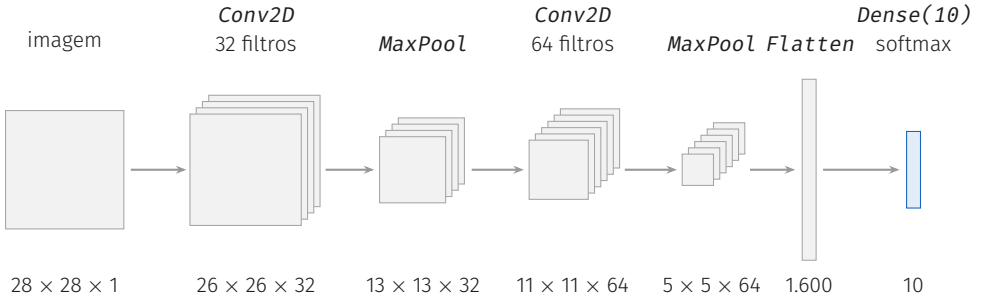
c) $13 \times 13 \times 16$; nenhum;

d) $52 \times 52 \times 32$; nenhum.

Resposta: a. O pooling divide a altura e a largura por 2, e age em cada um dos 32 mapas separadamente: continuam 32 (c está errada). Ele só escolhe o maior: não tem pesos (b). E encolhe, não aumenta (d).

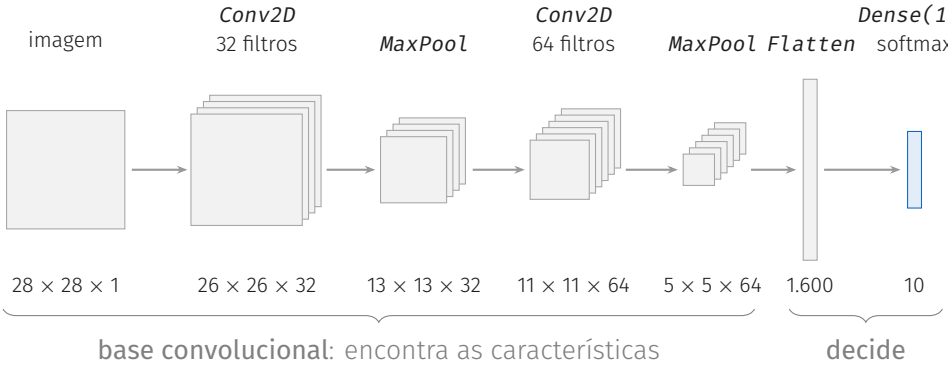
A CNN completa

A CNN, em uma figura



A imagem **encolhe** (28, 26, 13, 11, 5) e a pilha de mapas **cresce** (1, 32, 64): menos “onde”, mais “o quê”.

A CNN, em uma figura



A imagem **encolhe** (28, 26, 13, 11, 5) e a pilha de mapas **cresce** (1, 32, 64): menos “onde”, mais “o quê”. No fim, `Flatten` e uma camada densa com softmax: a classificação da Aula 3.

Agora vocês: sigam a imagem pela rede

Regras: *Conv2D* com filtro 3: tira 2 da altura e da largura. *MaxPool*: divide por 2, arredondando para baixo. *Flatten*: multiplica tudo.

Camada	Tamanho da saída	Parâmetros
entrada	$28 \times 28 \times 1$	—
Conv2D(32, 3)		
MaxPooling2D(2)		
Conv2D(64, 3)		
MaxPooling2D(2)		
Flatten		
Dense(10)		
	total:	

Agora vocês: sigam a imagem pela rede

Regras: *Conv2D* com filtro 3: tira 2 da altura e da largura. *MaxPool*: divide por 2, arredondando para baixo. *Flatten*: multiplica tudo.

Camada	Tamanho da saída	Parâmetros
entrada	$28 \times 28 \times 1$	—
Conv2D(32, 3)	$26 \times 26 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
MaxPooling2D(2)		
Conv2D(64, 3)		
MaxPooling2D(2)		
Flatten		
Dense(10)		
	total:	

Agora vocês: sigam a imagem pela rede

Regras: `Conv2D` com filtro 3: tira 2 da altura e da largura. `MaxPool`: divide por 2, arredondando para baixo. `Flatten`: multiplica tudo.

Camada	Tamanho da saída	Parâmetros
entrada	$28 \times 28 \times 1$	—
`Conv2D(32, 3)`	$26 \times 26 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
`MaxPooling2D(2)`	$13 \times 13 \times 32$	0
`Conv2D(64, 3)`		
`MaxPooling2D(2)`		
`Flatten`		
`Dense(10)`		
	total:	

Agora vocês: sigam a imagem pela rede

Regras: `Conv2D` com filtro 3: tira 2 da altura e da largura. `MaxPool`: divide por 2, arredondando para baixo. `Flatten`: multiplica tudo.

Camada	Tamanho da saída	Parâmetros
entrada	$28 \times 28 \times 1$	—
`Conv2D(32, 3)`	$26 \times 26 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
`MaxPooling2D(2)`	$13 \times 13 \times 32$	0
`Conv2D(64, 3)`	$11 \times 11 \times 64$	$(3 \times 3 \times 32 + 1) \times 64 = 18.496$
`MaxPooling2D(2)`		
`Flatten`		
`Dense(10)`		
	total:	

Agora vocês: sigam a imagem pela rede

Regras: *Conv2D* com filtro 3: tira 2 da altura e da largura. *MaxPool*: divide por 2, arredondando para baixo. *Flatten*: multiplica tudo.

Camada	Tamanho da saída	Parâmetros
entrada	$28 \times 28 \times 1$	—
Conv2D(32, 3)	$26 \times 26 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
MaxPooling2D(2)	$13 \times 13 \times 32$	0
Conv2D(64, 3)	$11 \times 11 \times 64$	$(3 \times 3 \times 32 + 1) \times 64 = 18.496$
MaxPooling2D(2)	$5 \times 5 \times 64$ ($11 \div 2 = 5{,}5 \to 5$)	0
Flatten		
Dense(10)		
	total:	

Agora vocês: sigam a imagem pela rede

Regras: *Conv2D* com filtro 3: tira 2 da altura e da largura. *MaxPool*: divide por 2, arredondando para baixo. *Flatten*: multiplica tudo.

Camada	Tamanho da saída	Parâmetros
entrada	$28 \times 28 \times 1$	—
Conv2D(32, 3)	$26 \times 26 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
MaxPooling2D(2)	$13 \times 13 \times 32$	0
Conv2D(64, 3)	$11 \times 11 \times 64$	$(3 \times 3 \times 32 + 1) \times 64 = 18.496$
MaxPooling2D(2)	$5 \times 5 \times 64$ $(11 \div 2 = 5{,}5 \rightarrow 5)$	0
Flatten	$5 \times 5 \times 64 = 1.600$	0
Dense(10)		
	total:	

Agora vocês: sigam a imagem pela rede

Regras: *Conv2D* com filtro 3: tira 2 da altura e da largura. *MaxPool*: divide por 2, arredondando para baixo. *Flatten*: multiplica tudo.

Camada	Tamanho da saída	Parâmetros
entrada	$28 \times 28 \times 1$	—
Conv2D(32, 3)	$26 \times 26 \times 32$	$(3 \times 3 \times 1 + 1) \times 32 = 320$
MaxPooling2D(2)	$13 \times 13 \times 32$	0
Conv2D(64, 3)	$11 \times 11 \times 64$	$(3 \times 3 \times 32 + 1) \times 64 = 18.496$
MaxPooling2D(2)	$5 \times 5 \times 64$ $(11 \div 2 = 5{,}5 \to 5)$	0
Flatten	$5 \times 5 \times 64 = 1.600$	0
Dense(10)	10	$(1.600 + 1) \times 10 = 16.010$
	total:	**34.826**

No Keras

```
modelo = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),           # altura, largura, canais
    layers.Conv2D(32, 3, activation="relu"),  # 32 filtros 3x3
    layers.MaxPooling2D(2),                   # blocos 2x2
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(2),
    layers.Flatten(),
    layers.Dense(10, activation="softmax"),   # 10 digitos
])
modelo.compile(optimizer="adam",
               loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.fit(X_train, y_train, epochs=5, batch_size=128, validation_split=0.1)
```

- `compile` e `fit`: os mesmos da Aula 3. Só a arquitetura mudou;
- `X_train`: $(60.000, 28, 28, 1)$ (1 canal). No `fit`, 54.000 treinam e 6.000 validam.

O resumo do modelo

```
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 32)          320
max_pooling2d (MaxPooling2D) (None, 13, 13, 32)            0
conv2d_1 (Conv2D)            (None, 11, 11, 64)       18,496
max_pooling2d_1 (MaxPooling2D) (None, 5, 5, 64)            0
flatten (Flatten)            (None, 1600)                  0
dense (Dense)                (None, 10)               16,010
=================================================================
Total params: 34,826
```

- *None*: o tamanho do lote (Aula 2). Pooling e *Flatten*: 0 parâmetros;

Saída simplificada do *modelo.summary()*.

O resumo do modelo

```
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 32)          320
max_pooling2d (MaxPooling2D) (None, 13, 13, 32)            0
conv2d_1 (Conv2D)            (None, 11, 11, 64)       18,496
max_pooling2d_1 (MaxPooling2D) (None, 5, 5, 64)            0
flatten (Flatten)            (None, 1600)                  0
dense (Dense)                (None, 10)               16,010
=================================================================
Total params: 34,826
```

- *None*: o tamanho do lote (Aula 2). Pooling e *Flatten*: 0 parâmetros;
- Quase metade dos parâmetros está na última camada, a densa, onde “tudo se liga com tudo”.

Saída simplificada do *modelo.summary()*.

Pergunta: de onde vem o 18.496?

A segunda camada, *`Conv2D(64, 3)`*, tem 18.496 parâmetros. De onde vem esse número?

a) $3 \times 3 \times 64$: 9 pesos por filtro;

b) $(3 \times 3 \times 32 + 1) \times 64$: cada filtro olha os 32 mapas da camada anterior;

c) $11 \times 11 \times 64$: um peso por número do mapa;

d) $(13 \times 13 \times 32 + 1) \times 64$.

Pergunta: de onde vem o 18.496?

A segunda camada, `Conv2D(64, 3)`, tem 18.496 parâmetros. De onde vem esse número?

a) $3 \times 3 \times 64$: 9 pesos por filtro;

b) $(3 \times 3 \times 32 + 1) \times 64$: cada filtro olha os 32 mapas da camada anterior;

c) $11 \times 11 \times 64$: um peso por número do mapa;

d) $(13 \times 13 \times 32 + 1) \times 64$.

Resposta: b. A entrada dessa camada é a pilha de 32 mapas: cada filtro tem $3 \times 3 \times 32 = 288$ pesos, mais 1 bias, e são 64 filtros. (a) esquece os 32 mapas; (c) e (d) usam o tamanho do mapa, que não entra na conta.

Resultado: rede densa ou CNN?

O mesmo MNIST, o mesmo treino (Adam, 5 épocas, lote 128). Teste: 10.000 dígitos que a rede nunca viu.

Rede	Parâmetros	Acerto no teste	Erros em 10.000
Densa: `Dense(128)`, `Dense(10)`	101.770	97,3%	269

Nosso experimento, com o Keras.

Resultado: rede densa ou CNN?

O mesmo MNIST, o mesmo treino (Adam, 5 épocas, lote 128). Teste: 10.000 dígitos que a rede nunca viu.

Rede	Parâmetros	Acerto no teste	Erros em 10.000
Densa: `Dense(128)`, `Dense(10)`	101.770	97,3%	269
CNN do slide anterior	34.826	98,9%	113

Nosso experimento, com o Keras.

Resultado: rede densa ou CNN?

O mesmo MNIST, o mesmo treino (Adam, 5 épocas, lote 128). Teste: 10.000 dígitos que a rede nunca viu.

Rede	Parâmetros	Acerto no teste	Erros em 10.000
Densa: `Dense(128)`, `Dense(10)`	101.770	97,3%	269
CNN do slide anterior	34.826	98,9%	113

- A CNN tem **3 vezes menos** parâmetros e erra **menos da metade**;
- Em fotos de verdade, maiores e mais variadas que o MNIST, a diferença é ainda maior, por isso as CNNs se tornaram uma das principais arquiteturas para visão computacional.

Nosso experimento, com o Keras.

Palpite: a CNN e os pixels embaralhados



Palpite, antes de ver o resultado

A rede densa nem percebeu os pixels embaralhados: 97,3% $\rightarrow$ 97,2%. E a CNN?

Palpite: a CNN e os pixels embaralhados



Palpite, antes de ver o resultado

A rede densa nem percebeu os pixels embaralhados: 97,3% $\rightarrow$ 97,2%. E a CNN?

	Imagens normais	Embaralhadas
Densa	97,3%	97,2%
CNN	98,9%	95,9%

Palpite: a CNN e os pixels embaralhados



Palpite, antes de ver o resultado

A rede densa nem percebeu os pixels embaralhados: 97,3% → 97,2%. E a CNN?

	Imagens normais	Embaralhadas
Densa	97,3%	97,2%
CNN	98,9%	95,9%

A CNN **piora**: o embaralhamento destrói os padrões locais originais, dificultando o trabalho dos filtros.

Fechamento

Pergunta final: por que a CNN foi melhor?

Para discutir

No MNIST, a CNN teve menos parâmetros e menos erros que a rede densa. O que a CNN aproveita na imagem que a rede densa não aproveita diretamente?

Pergunta final: por que a CNN foi melhor?

Para discutir

No MNIST, a CNN teve menos parâmetros e menos erros que a rede densa. O que a CNN aproveita na imagem que a rede densa não aproveita diretamente?

- **Vizinhança**: os filtros combinam pixels próximos para encontrar padrões locais;
- **Compartilhamento**: o mesmo filtro é aplicado em diferentes lugares;
- **Composição**: camadas sucessivas combinam mapas para representar padrões mais complexos.

Atenção: a CNN não é sempre melhor. A escolha depende dos dados, da tarefa e da avaliação.

Agora vocês: três perguntas para fechar

1. Uma imagem tem formato `(64, 64, 3)`. O que representa o 3?
2. Por que a camada `Conv2D(32, 3)` aplica os mesmos filtros em diferentes regiões?
3. Se embaralharmos os pixels **da mesma maneira em todas as imagens**, o que esperamos que aconteça com uma CNN treinada nesses dados?

Agora vocês: três perguntas para fechar

1. Uma imagem tem formato `(64, 64, 3)`. O que representa o 3?
2. Por que a camada `Conv2D(32, 3)` aplica os mesmos filtros em diferentes regiões?
3. Se embaralharmos os pixels **da mesma maneira em todas as imagens**, o que esperamos que aconteça com uma CNN treinada nesses dados?

Respostas esperadas

1. Os **três canais** de cor: vermelho, verde e azul;
2. Para detectar um mesmo padrão **independentemente de sua posição**;
3. Ela tende a **perder a vantagem dos padrões locais originais**; é preciso medir no teste.

O que levar desta aula

1. **Imagens são tensores:** altura $\times$ largura $\times$ canais;
2. **Convolução:** filtros pequenos percorrem a imagem, compartilham pesos e produzem mapas de características;
3. **Pooling:** reduz a resolução dos mapas, preservando respostas fortes;
4. **Uma CNN pode ser compacta:** contamos seus parâmetros e montamos uma no Keras;
5. **Avaliar importa:** comparamos a CNN a uma rede densa e investigamos o que acontece ao embaralhar pixels.

Na próxima aula

Uma CNN pode aprender a classificar imagens, mas ainda precisamos perguntar:

- Como evitar que memorize as fotos do treinamento?
- Como treinar quando temos poucas imagens rotuladas?
- Como investigar **o que influenciou** uma previsão?

Aula 5: generalização e interpretabilidade

Regularização, aumentação de dados, transferência de aprendizado e Grad-CAM.

Referências

- CHOLLET, F. *Deep Learning with Python*. 2. ed. Manning, 2021. Cap. 8.
- PRINCE, S. J. D. *Understanding Deep Learning*. MIT Press, 2023. Cap. 10 (redes convolucionais).
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016. Cap. 9.
- LECUN, Y. et al. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.
- KARPATHY, A. et al. CS231n: Convolutional Neural Networks for Visual Recognition. Notas de aula, Stanford.